Implementation of 4x1 Multiplexer

Introduction

This lab provides an introduction to designing a simple 4x1 multiplexer using Verilator. You will learn to create the design, write a corresponding testbench, simulate the behavior, and analyze the output using waveform tools.

Objectives

After completing this lab, you will be able to:

- Write Verilog code for a simple 4x1 multiplexer in Dataflow, Behavior and Gate level modelling in verilog.
- Develop a testbench to verify the design.
- Simulate the design using Verilator.
- Observe and analyze the 4x1 multiplexer behavior using waveform output.

Procedure:

This lab consists of the following steps:

1. Creating the lab directory
2. Creating new project directory
3. Writing the design file
4. Writing testbench file
5. Simulating the design using Verilator
6. Analyzing the waveform dump file

1) Gate-Level Modelling

Step 1: Navigate back to the Lab Directory

1.1: Create a new directory inside your lab workspace:

`mkdir verilator_lab_project,` **and if lab directory already available**

1.2: Navigate to the lab directory:

```
cd verilator_lab_project
```

1.2 Create a new project directory for 4x1 MUX in gate-level modelling

```
mkdir mux4x1_gatelevel
```

1.3 Navigate into the project directory

```
cd mux4x1_gatelevel
```

Step 2: Create the Design File

2.1 Open gvim to write the 4x1 MUX design in gate-level modelling

```
gvim mux4x1_gatelevel.v
```

```
// 4x1 Multiplexer - Gate Level Modeling using basic gates
`timescale 1ns/1ps

module mux4x1_gatelevel (
  input [3:0] in,      // 4-bit input vector
  input [1:0] sel,     // 2-bit select input
  output out           // Output
);

wire not_s0, not_s1;
wire and0, and1, and2, and3;

// Inverter gates
not (not_s0, sel[0]);
not (not_s1, sel[1]);

// AND gates
and (and0, not_s1, not_s0, in[0]);
and (and1, not_s1, sel[0], in[1]);
and (and2, sel[1], not_s0, in[2]);
and (and3, sel[1], sel[0], in[3]);

// OR gate
or (out, and0, and1, and2, and3);

endmodule
```

2.3 Save and exit the editor

```
:wq!
```

Step 3: Create the Testbench File

3.1 Open GVim

```
gvim mux_gatelevel_tb.v
```

3.2 Testbench Code

```
// Testbench for 4x1 Multiplexer - Gate Level Modeling
`timescale 1ns/1ps

module mux4x1_gatelevel_tb;

reg [3:0] in;
reg [1:0] sel;
wire out;
```

```
// Instantiate the Unit Under Test
mux4x1_gatelevel uut (
  .in(in),
  .sel(sel),
  .out(out)
);

initial begin

$display("GATE LEVEL MUX4x1 TEST");
$display("SEL | IN     | OUT");
$display("---------------------");

in = 4'b1101;

sel = 2'b00; #10 $display("%b | %b | %b", sel, in, out);
sel = 2'b01; #10 $display("%b | %b | %b", sel, in, out);
sel = 2'b10; #10 $display("%b | %b | %b", sel, in, out);
sel = 2'b11; #10 $display("%b | %b | %b", sel, in, out);

$finish;

end

initial begin
$dumpfile("mux4x1_gatelevel.vcd");
$dumpvars(0, mux4x1_gatelevel_tb);
end

endmodule
```

3.3 Save and exit

```
:wq!
```

3.4 Check files using the command : `ls`

Step 4: Simulate the Design Using Verilator

4.1 Compile

```
verilator --binary -j 0 -Wall mux4x1_gatelevel.v \
mux_gatelevel_tb.v --top mux4x1_gatelevel_tb --timing \
--CFLAGS "-std=c++20" --trace
```

4.2 Fix errors if any and rerun

4.3 Check directory **obj_dir** : `ls`

4.4 Go to object directory : `cd obj_dir`

4.5 Build simulation

```
make -f Vmux4x1_gatelevel_tb.mk Vmux4x1_gatelevel_tb
```

4.6 Run simulation

```
./Vmux4x1_gatelevel_tb
```

4.7 Observe terminal output

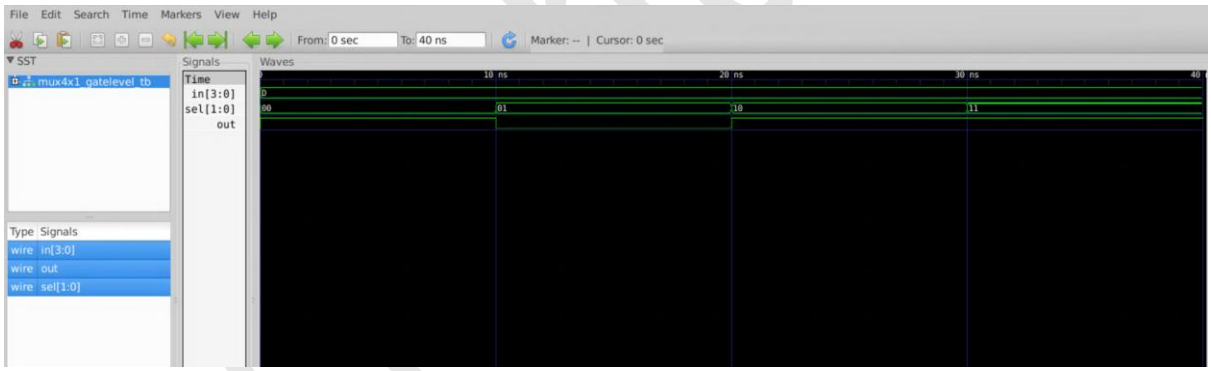
```
GATE LEVEL MUX4x1 TEST
SEL | IN     | OUT
--------------------
00 | 1101 | 1
01 | 1101 | 0
10 | 1101 | 1
11 | 1101 | 1
- mux_gatelevel_tb.v:30: Verilog $finish
```

Step 5: View Waveform

5.1 Open GTKWave

```
gtkwave mux4x1_gatelevel.vcd
```

5.2 Observe the waveform of the MUX.



2) Dataflow Modelling

Step 1: Create a Lab Directory

1.3: Create a new project directory for 4x1 mux in dataflow style of modelling

```
mkdir mux4x1_dataflow
```

1.4: Navigate into the project directory for 4x1 mux in dataflow style of modelling

```
cd mux4x1_dataflow
```

Step 2: Create the Design File

2.1: Open GVim to write the 4x1 mux design in dataflow style of modelling

```
gvim mux4x1_dataflow.v
```

// 4x1 Multiplexer - Dataflow Modelling using logic equation

```
`timescale 1ns/1ps // Time unit = 1ns, time precision = 1ps
module mux4x1_dataflow(
  input [3:0] in,          // 4-bit input lines
  input [1:0] sel,         // 2-bit select lines
  output out               // Output of the MUX
);
  // Dataflow modeling of 4:1 multiplexer
  assign out = (~sel[1] & ~sel[0] & in[0]) |
               (~sel[1] &  sel[0] & in[1]) |
               ( sel[1] & ~sel[0] & in[2]) |
               ( sel[1] &  sel[0] & in[3]);
endmodule
```

2.3: Save and exit the editor

Type `:wq!` and press Enter

Step 3: Create the testbench File

3.1: Open GVim to create the testbench

```
gvim mux4x1_dataflow_tb.v
```

3.2: Write the testbench code to test the 4x1 mux behavior

// Testbench for 4x1 mux - Dataflow Modeling

```
`timescale 1ns/1ps
module mux4x1_dataflow_tb;
  // Declare testbench variables
  reg [3:0] in;       // 4-bit input vector
  reg [1:0] sel;      // 2-bit selector
  wire out;           // Output from the multiplexer
  // Instantiate the Unit Under Test (UUT)
  mux4x1_dataflow uut (
    .in(in),
    .sel(sel),
```

```
    .out(out)
  );
  // Test stimulus block
  initial begin
    $display("DATAFLOW 4x1 MUX TEST");
    $display("----------------------");
    $display("SEL | IN     | OUT");
    $display("----------------------");
    // Set input values
    in = 4'b1010; // You can change this pattern to test more
combinations
    // Apply all select line combinations with delay
    sel = 2'b00; #10 $display("%b   | %b |  %b", sel, in, out);
    sel = 2'b01; #10 $display("%b   | %b |  %b", sel, in, out);
    sel = 2'b10; #10 $display("%b   | %b |  %b", sel, in, out);
    sel = 2'b11; #10 $display("%b   | %b |  %b", sel, in, out);
    $finish; // End simulation
  end
initial begin
     $dumpfile("mux4x1_dataflow.vcd");
     $dumpvars(0, mux4x1_dataflow_tb);
end
endmodule
```

3.3: Save and exit the editor

Type `:wq!` and press Enter

3.4: List the files to confirm they are saved

```
ls
```

Step 4: Simulate the Design Using Verilator

4.1: Run Verilator to compile the design and testbench

```
verilator --binary -j 0 -Wall mux4x1_dataflow.v \
mux4x1_dataflow_tb.v --top mux4x1_dataflow_tb --timing \
--CFLAGS "-std=c++20" --trace
```

4.2: If any errors occur, resolve them and rerun the command.

4.3: After successful compilation, a new directory obj_dir will be created

```
ls
```

4.4: Navigate into the `obj_dir`

```
cd obj_dir
```

4.5: Build the simulation binary

```
make -f Vmux4x1_dataflow_tb.mk Vmux4x1_dataflow_tb
```

4.6: After compilation, an executable named `Vmux4x1_dataflow_tb` will be generated

4.7: Run the simulation

```
./Vmux4x1_dataflow_tb
```

4.8: Observe the simulation output in the terminal

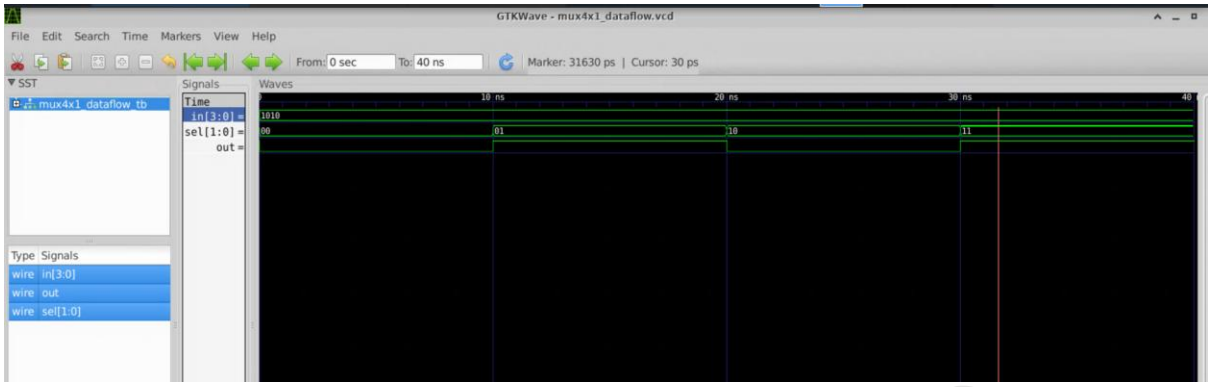
```
DATAFLOW 4x1 MUX TEST
----------------------
SEL | IN     | OUT
----------------------
00  | 1010 |  0
01  | 1010 |  1
10  | 1010 |  0
11  | 1010 |  1
- mux4x1_dataflow_tb.v:27: Verilog $finish
```

Step 5: View Waveform Output

5.1: Launch GTKWave to view the waveform file

```
gtkwave mux4x1_dataflow.vcd
```

5.2: Examine the output of mux in waveform



3) Behavioral Modelling

Steps:

Step 1: Navigate back to the Lab Directory

1.1: Navigate to the lab directory:

```
cd ~/verilator_lab_project
```

1.2: Create a new project directory for 4x1 mux in behavioral style of modelling

```
mkdir mux4x1_behavioral
```

1.3: Navigate into the project directory for 4x1 mux in behavioral style of modelling

```
cd mux4x1_behavioral
```

Step 2: Create the Design File

2.1: Open GVim to write the 4x1 mux design in behavioral style of modelling

```
gvim mux4x1_behavioral.v
```

Code:

// 4x1 Multiplexer - Behavioral Modeling

// Implements case statement based selection logic

```
`timescale 1ns/1ps  // Time unit = 1ns, time precision = 1ps
module mux4x1_behavioral (
  input [3:0] in,        // 4-bit input bus
  input [1:0] sel,       // 2-bit select input
  output reg out         // Single output bit
);
  always @(*) begin          // Combinational logic block
```

```
case (sel)
      2'b00: out = in[0];                // Select input 0
      2'b01: out = in[1];                // Select input 1
      2'b10: out = in[2];                // Select input 2
      2'b11: out = in[3];                // Select input 3
      default: out = 1'bx;               // Undefined case
    endcase
  end
endmodule
```

2.3: Save and exit the editor

Type `:wq!` and press Enter

Step 3: Create the testbench File

3.1: Open GVim to create the testbench

```
gvim mux4x1_behavioral_tb.v
```

3.2: Write the testbench code to test the 4x1 mux behavior

// Testbench for 4x1 mux - Behavioral Modeling

```
`timescale 1ns/1ps  // Time unit = 1ns, time precision = 1ps
module mux4x1_behavioral_tb;
  // Declare testbench variables
  reg [3:0] in;       // 4-bit input vector
  reg [1:0] sel;      // 2-bit selector
  wire out;           // Output from the multiplexer
  // Instantiate the Unit Under Test (UUT)
  mux4x1_behavioral uut (
    .in(in),
    .sel(sel),
    .out(out)
  );
  // Test stimulus block
  initial begin
```

```
    $display("BEHAVIORAL 4x1 MUX TEST");
    $display("----------------------");
    $display("SEL | IN     | OUT");
    $display("----------------------");
    in = 4'b1010;  // Input pattern
    sel = 2'b00; #10 $display("%b  | %b |  %b", sel, in, out);
    sel = 2'b01; #10 $display("%b  | %b |  %b", sel, in, out);
    sel = 2'b10; #10 $display("%b  | %b |  %b", sel, in, out);
    sel = 2'b11; #10 $display("%b  | %b |  %b", sel, in, out);
    $finish;       // End simulation
  end
  // Dump waveform data for GTKWave
  initial begin
    $dumpfile("mux4x1_behavioral.vcd");     // VCD output file
    $dumpvars(0, mux4x1_behavioral_tb);  // Dump all variables
in this module
  end
endmodule
```

3.3: Save and exit the editor

Type `:wq!` and press Enter

3.4: List the files to confirm they are saved

```
ls
```

Step 4: Simulate the Design Using Verilator

4.1: Run Verilator to compile the design and testbench

```
verilator --binary -j 0 -Wall mux4x1_behavioral.v \
mux4x1_behavioral_tb.v --top mux4x1_behavioral_tb --timing \
--CFLAGS "-std=c++20" --trace
```

4.2: If any errors occur, resolve them and rerun the command.

4.3: After successful compilation, a new directory obj_dir will be created

```
ls
```

4.4: Navigate into the obj_dir

```
cd obj_dir
```

4.5: Build the simulation binary

```
make -f Vmux4x1_behavioral_tb.mk Vmux4x1_behavioral_tb
```

4.6: After compilation, an executable named `Vmux4x1_behavioral_tb` will be generated

4.7: Run the simulation

```
./Vmux4x1_behavioral_tb
```

4.8: Observe the simulation output in the terminal

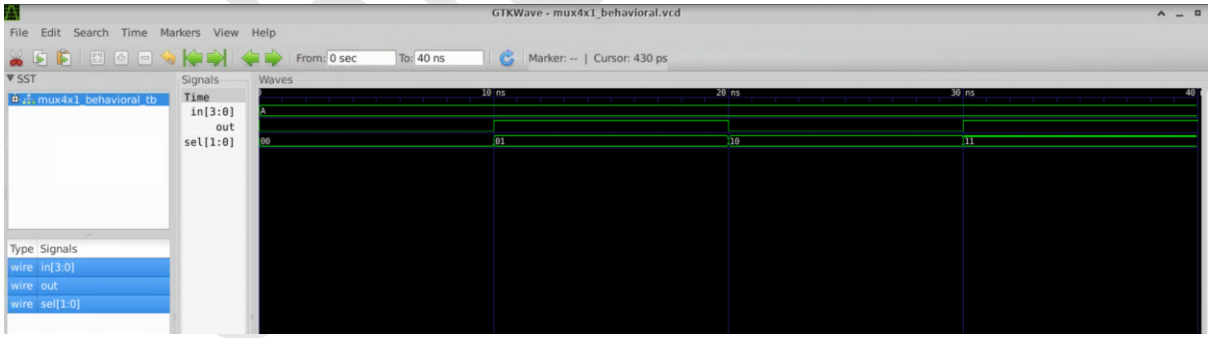
```
BEHAVIORAL 4x1 MUX TEST
----------------------
SEL | IN     | OUT
----------------------
00  | 1010 |  0
01  | 1010 |  1
10  | 1010 |  0
11  | 1010 |  1
- mux4x1 behavioral tb.v:26: Verilog $finish
```

Step 5: View Waveform Output

5.1: Launch GTKWave to view the waveform file

```
gtkwave mux4x1_behavioral.vcd
```

5.2: Examine mux 4x1 output using GTKwave.



Conclusion

In this lab, you implemented and simulated a simple 4x1 multiplexer using verilator in all the Verilog modelling styles. By analyzing the simulation and waveform output, you observed the behavior of 4x1 multiplexer. This lab enhanced your understanding of basic combinational logic design and open-source digital simulation tools.